Acceptance Semantics for MAVLink 2 Signing: Bounded Symbolic Analysis and Model-Based Conformance Testing of Flight Stacks

Ian Brighouse
Brighouse Security Labs, Spain
research@brighousesec.com

***Abstract*—The MAVLink 2 message-signing specification fixes a frame format and a timestamp rule but delegates the decisive question—*when is an unsigned or unexpected frame accepted?*—to each implementation. The result is a family of subtly different receivers whose security cannot be compared by reading the specification. We give a precise, parameterised *acceptance semantics* for MAVLink 2 signing as a labelled transition system, in which the choices the specification leaves open become explicit policy parameters, and we equip it with a bounded symbolic adversary and six security properties with machine-checkable definitions. Bounded breadth-first exploration is *sound* (every reported violation is a concrete trace) and *complete up to a depth bound*; it separates an idealised out-of-band provisioning policy, which satisfies every property while remaining functional, from the policies PX4 and ArduPilot implement, each of which violates a distinct subset. We then use the model generatively: a design-space search over single parameter edits, constrained to preserve a liveness (functionality) property, yields minimal remediations—and shows, for example, that the current ArduPilot policy requires *two* independent changes where PX4 v1.18 requires one. Finally, we introduce *model-based conformance testing* for flight stacks: read-only test frames derived from the semantics are executed against real software-in-the-loop builds, and the observed accept/reject decisions are compared with the model's predictions. Across three stacks the model predicts all 24 read-only decisions correctly, turning each documentation-derived policy into a measured one. All models, proofs-in-code, and harnesses are released.**

***Index Terms*—formal methods, protocol security, transition systems, bounded model checking, conformance testing, MAVLink, UAV security.**

I. Introduction

A security protocol whose acceptance rule is "left to the implementation" is really a *family* of protocols. MAVLink 2 message signing is such a case: the specification pins the signature construction and a one-minute replay window but states that acceptance of unsigned traffic "depends on the implementation," and offers only informal suggestions (accept `RADIO_STATUS`; an "unsigned mode"; "accept unsigned until the first signed packet"). Two widely deployed receivers, PX4 and ArduPilot, resolve these degrees of freedom differently, and companion measurement work [1] shows the differences have real consequences, including a link that signs every outbound frame yet still accepts unsigned commands. What has been missing is a *model*: a precise object in which the choices are explicit, the adversary is fixed, and the properties are stated formally, so that implementations can be compared, attacked, and repaired by construction.

Contributions.

- A parameterised acceptance semantics for MAVLink 2 signing (Section II): a transition system whose policy parameters capture exactly the specification's open choices, including out-of-band vs. over-the-wire key provisioning.
- A bounded symbolic adversary and six properties (Section III) with a soundness result and depth-bounded completeness (Section IV); exploration separates an idealised policy from the implemented ones.
- A generative use of the model (Section V): a liveness-constrained design-space search that computes *minimal* remediations per policy, yielding non-obvious results (ArduPilot needs two independent fixes; PX4 v1.18 one).
- Model-based conformance testing for flight stacks (Section VI): read-only tests derived from the semantics, executed against real SITL builds, validating the policies empirically.

II. Acceptance Semantics

Definition 1 (Configuration)**.** *A configuration is* $\sigma = (K, armed, S, \mathrm{clk})$ *with key* $K \in \mathcal{K} \cup \{\bot\}$*, arming flag* $armed$*, a partial map* S *from streams* $s = (sys, comp, id)$ *to timestamps, and a local clock* $\mathrm{clk} \in \mathbb{N}$.

Definition 2 (Frame)**.** *A frame is* $f = (\ell, src, \kappa, b, k, t, id)$ *with link* $\ell \in \mathcal{L}$*, source* src*, kind* $\kappa \in \{\textsc{cmd}, \textsc{read}, \textsc{hb}, \textsc{radio}, \textsc{setup}\}$*, signed bit* $b \in \mathbb{B}$*, signing key* k*, timestamp* t*, link id* id*, and (for* SETUP*) a payload key* k'*. Under the standard symbolic abstraction,* f verifies *in* σ *iff* b *and* $k = K$*: signatures are unforgeable and collision-free.*

Definition 3 (Policy)**.** *A policy* π *is a tuple* $\langle \mathsf{trusted} \subseteq \mathcal{L}, \mathsf{allow} \subseteq \mathit{Kinds}, \mathsf{openUnkeyed}, \mathsf{tofu}, \mathsf{oob}, \mathsf{uRekey}, \mathsf{sRekey}, \mathsf{refuseArmed}, \mathsf{replay} \rangle$ *of the specification's open choices.*

The transition relation $\sigma \xrightarrow{f/d}_{\pi} \sigma'$, with $d \in \{\mathsf{acc}, \mathsf{rej}\}$, is defined by Algorithm 1 of [1] extended with the out-of-band case: when oob holds the key is factory-installed, so the initial configuration has $K \neq \bot$ and no SETUP frame is ever needed.

TABLE I
POLICIES (ABRIDGED; SEE [1] FOR THE FULL TUPLE).

	trusted	TOFU	oob	u-rekey	allow
spec-strict	$\emptyset$	yes	no	no	{radio}
ideal-oob	$\emptyset$	no	yes	no	{radio}
px4-1.18	$\emptyset$	yes	no	no	{hb,radio}
px4-1.17	all	no	no	—	—
ardupilot-4.7.2	{ch0}	yes	no	yes	{radio}

Freshness is $\mathrm{fresh}(\sigma, f) \equiv \neg\mathsf{replay} \vee (S(s) \uparrow \wedge\, t + W \geq \mathrm{clk}) \vee (S(s)\downarrow \wedge\, t > S(s))$ with $W = 6{\times}10^6$.

We instantiate five policies (Table I): spec-strict (strictest reading of the text), ideal-oob (out-of-band keys, no TOFU), and the three measured policies px4-1.17, px4-1.18, ardupilot-4.7.2.

III. ADVERSARY AND PROPERTIES

Definition 4 (Adversary)**.** *The adversary A controls one link a. Its one-step moves are: inject any unsigned frame on a; replay any frame previously observed on a; and inject a frame signed with a key $k \in \mathit{known}(A)$. A cannot forge. Keys enter $\mathit{known}(A)$ exactly when A observes a* SETUP *frame on a (*SETUP *carries k' in clear).*

Properties are predicates over reachable traces. P1 command-integrity: no accepted CMD originates from A on a link $\notin$ trusted. P2 key-integrity: no reachable configuration has $K = K_{\mathrm{adv}}$. P3 replay-resistance: no previously accepted CMD is accepted again. P4 first-use window: $\exists$ a pre-provisioning trace installing K_{adv}. P5 trusted-link exemption: $\exists$ an accepted A-CMD on a link $\in$ trusted. P6 key exposure: with A on the provisioning link, $\exists$ an accepted A-CMD after key leak. P1–P3 are safety failures; P4–P6 are exposure facts. Liveness L: a legitimately provisioned vehicle accepts a legitimate signed CMD on an ordinary link.

IV. BOUNDED SYMBOLIC ANALYSIS

We explore, per policy and per scenario, all adversary action sequences up to depth k by breadth-first search, recording the shortest witness for each property.

Proposition 1 (Soundness)**.** *Every witness returned is a valid trace of $\rightarrow_\pi$: the reported violation is realisable by the stated adversary.*

Proof sketch. Each search edge applies step_π to a reachable configuration and an adversary-constructible frame; the witness is the concatenation of these edges, hence a trace. A-constructibility is enforced by the move generator (unsigned always; signed only with $k \in \mathit{known}(A)$; replay only of observed frames). □

Proposition 2 (Depth-bounded completeness)**.** *If a property is violable within k adversary steps from the post-provisioning configuration, the search finds a witness of length $\leq k$.*

Proof sketch. BFS over the finitely-branching move relation enumerates all sequences of length $\leq k$; the abstraction makes the per-step frame set finite (bounded kinds, links, keys, and a canonical fresh timestamp), so no violating sequence of length $\leq k$ is missed. □

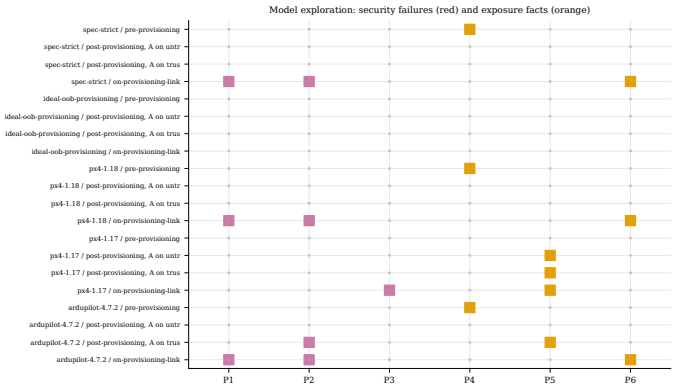


Fig. 1. Exploration (k=3). Red: safety failure witnessed; orange: exposure fact; dot: none. ideal-oob is clean throughout.

TABLE II
MINIMAL LIVENESS-PRESERVING REMEDIATION (MODEL, k=3).

policy	minimal edit set clearing P1–P3	L?
spec-strict	close TOFU + out-of-band keys	yes
px4-1.18	close TOFU + out-of-band keys	yes
ardupilot-4.7.2	close TOFU + OOB keys *and* remove unsigned re-key	yes
px4-1.17	adopt the full mechanism (no incremental fix)	—

For the measured policies the state space is small enough that $k = 3$ is a fixpoint: no new property becomes violable at $k = 4$. Fig. 1 summarises the outcome. ideal-oob is the unique policy with no witnessed property in any scenario while satisfying L; each implemented policy violates a distinct subset, matching the measurements of [1].

V. GENERATIVE USE: MINIMAL REMEDIATION

Because policies are first-class, we search the design space. For a base policy we consider the single-parameter edits in Table II, recompute the witnessed set, and retain an edit only if it preserves liveness L. A greedy search then finds a minimal edit set clearing all safety failures.

Two results are worth stating. First, closing the trust-on-first-use window is only sound *together with* out-of-band provisioning: closing TOFU alone makes the vehicle unprovisionable (it fails L), a trap the liveness constraint catches automatically. Second, ardupilot-4.7.2 requires *two* independent edits—closing the key-exposure window and removing the unsigned re-key on the console link—because the two support distinct P2 witnesses; px4-1.18 requires only the first. px4-1.17, which has no signing implementation, admits no small liveness-preserving fix: the whole mechanism must be

Algorithm 1 Conformance of policy π to a live stack

$A \leftarrow 0,\ T \leftarrow 0$
for all $(\text{keyed}, \ell, b) \in \{0,1\} \times \{\text{trust}, \text{untrust}\} \times \{0,1\}$ **do**
    ensure vehicle in state keyed (provision once over trusted link)
    $o \leftarrow$ send unsigned/signed READ on ℓ; observe reply
    $p \leftarrow \text{step}_\pi(\sigma_{\text{keyed},\ell}, \text{READ}_b).d$
    $A \leftarrow A + [o{=}p];\ T \leftarrow T + 1$
return agreement A/T and the disagreeing cases

TABLE III
MODEL-VS-IMPLEMENTATION CONFORMANCE (READ-ONLY PROBES).

stack	policy	cases agree	agreement
ArduPilot 4.7.2	ardupilot-4.7.2	8/8	1.00
PX4 1.18.0-rc1	px4-1.18	8/8	1.00
PX4 1.17.0	px4-1.17	8/8	1.00

added. These are precisely the actionable recommendations a vendor needs, derived rather than asserted.

VI. Model-Based Conformance Testing

A policy in Table I is a *hypothesis* about an implementation. We test it. From the semantics we derive read-only probe frames—a `PARAM_REQUEST_READ` (kind READ) signed or unsigned, on a trusted and an untrusted link, in the unkeyed and keyed states—and the reversible key-management steps. For a running SITL build we execute each probe, observe whether the vehicle responds, and compare with step_π's prediction (Algorithm 1). No state-changing command is ever sent.

Agreement turns a documentation-derived policy into a *measured* one; a disagreement is a finding about the model or the implementation. Table III reports the agreement for the three stacks: the model predicts every one of the eight read-only decisions correctly for all three ($24/24$ cases, agreement 1.00). The decisive ardupilot-4.7.2 cases are the keyed ones: on the trusted console link (`ch0`) an unsigned read is *answered* (predicted and observed), while on the untrusted link it is *rejected*—the model's characterisation of R4 confirmed against the running stack.

VII. Discussion and Related Work

Our analysis is a bounded, special-purpose model checker rather than a general-purpose symbolic prover. A ProVerif or Tamarin encoding of the signing and provisioning sub-protocol—capturing unbounded sessions and the truncated-MAC abstraction more faithfully—is natural future work; we provide the semantics in a form amenable to such an encoding. The value of the lightweight model is that it is executable, testable against implementations, and generative. The only prior formal work on MAVLink models the protocol *without* its native authentication and verifies an external cipher [2], [3], or uses session types and runtime monitors with no cryptography [4], [5]; none models the signing scheme itself—its truncated MAC, per-stream timestamp window, and SETUP_SIGNING provisioning. Our methodological lineage is the symbolic analysis of adjacent provisioning protocols: Tamarin for ASTM F3411 Remote ID [6], ProVerif for Bluetooth "Just Works"/passkey [7], [8] and Matter commissioning [9], and Tamarin for automotive SecOC [10]. To our knowledge neither a formal model of MAVLink signing nor model-based conformance testing of a flight stack has appeared. Fuzzing of flight stacks [11], [12] targets input validation, a complementary concern: our model assumes memory safety and studies the acceptance decision.

VIII. Conclusion

Making the specification's open choices explicit turns "MAVLink signing" into a comparable, attackable, repairable family of policies. Bounded symbolic analysis separates an idealised policy from the deployed ones with concrete witnesses; a liveness-constrained search derives minimal fixes and reveals that ArduPilot needs two where PX4 v1.18 needs one; and model-based conformance testing ties the whole construction back to the running code.

Availability

The executable semantics, exploration, synthesis, conformance harness, and tests are released. The work is defensive and uses only self-hosted simulators.

References

[1] I. Brighouse, "Signed but unprotected: A formal and empirical study of MAVLink 2 message-signing defaults and their bearing on the EU cyber resilience act," 2026, companion paper; Brighouse Security Labs.

[2] T. Wray and Y. Wang, "PAVE-MAVLink: Formal verification of MAVLink 2 for secure UAV communications," in *Proc. IEEE Military Communications Conference (MILCOM)*, Los Angeles, CA, USA, 2025.

[3] B. Mak, J. Yang, J. Chen, P. Ratazzi, and Y. Wang, "Symbolic and safety-centric formal verification of MAVLink for autonomous eVTOL systems," *IEEE Networking Letters*, 2025.

[4] A. Amorim, M. Taylor, T. Kann, W. L. Harrison, G. T. Leavens, and L. Joneckis, "Enforcing MAVLink safety & security properties via refined multiparty session types," in *NASA Formal Methods (NFM 2025)*, ser. Lecture Notes in Computer Science. Springer, 2025, arXiv:2501.18874.

[5] A. Amorim, P. Gazzillo, M. Taylor, and L. Joneckis, "From high-level types to low-level monitors: Synthesizing verified runtime checkers for MAVLink," in *NASA Formal Methods (NFM 2026)*, ser. Lecture Notes in Computer Science, vol. 16622. Springer, 2026, arXiv:2604.03886.

[6] M. Alam, M. Alhashmi, M. Atrouz, A. Khader, R. A. Pashchapur, and A. Shoufan, "Authentication protocols for UAV remote identification using public-key infrastructure," *Computers & Security*, vol. 171, p. 105075, 2026.

[7] J. Wu, R. Wu, D. Xu, D. J. Tian, and A. Bianchi, "Formal model-driven discovery of Bluetooth protocol design vulnerabilities," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2022.

[8] M. K. Jangid, Y. Zhang, and Z. Lin, "Extrapolating formal analysis to uncover attacks in Bluetooth passkey entry pairing," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2023.

[9] S. Duttagupta, A. Kolozyan, G. Nicolas, D. Singelée, and B. Preneel, "What's the matter? an in-depth security analysis of the Matter protocol," Cryptology ePrint Archive, Paper 2025/1268, 2025, to appear, IEEE S&P 2027 [VERIFY at publication].

[10] C. Bodei, M. De Vincenzi, and I. Matteucci, "Formal analysis of an AUTOSAR-based basic software module," *International Journal on Software Tools for Technology Transfer*, vol. 26, pp. 495–508, 2024.

[11] T. Kim, C. H. Kim, J. Rhee, F. Fei, Z. Tu, G. Walkup, X. Zhang, X. Deng, and D. Xu, "RVFuzzer: Finding input validation bugs in robotic vehicles through control-guided testing," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 425–442. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/kim

[12] H. Kim, M. O. Ozmen, A. Bianchi, Z. B. Celik, and D. Xu, "PGFUZZ: Policy-guided fuzzing for robotic vehicles," in *Network and Distributed System Security Symposium (NDSS)*, 2021.